P3 – Dataset as a Product

IDIG4120 – Digital Information Processing

Authors: Ole Marius Glomsrud and Vinjar Aurstad Berland
Date: 7 November 2026

1. Introduction

This report documents the dataset and processing pipeline used in our lossless image compression project. The aim of P3 is to make the dataset understandable, verifiable, and reproducible for other users.

The project builds on the Huffman compression codec developed in P2. In P3, the codec is applied to a small image dataset, with additional quality assurance, integrity verification, and reproducibility measures.

2. Dataset Datasheet

2.1. Identity and Purpose

Dataset: Pistachio Image Dataset
Subset: Kirmizi_Pistachio
Project version: P3, version 1.0
Owners: Ole Marius Glomsrud and Vinjar Aurstad Bergland

The dataset contains images of pistachio nuts. For this project, the images are used to evaluate a lossless compression pipeline based on canonical Huffman coding and optional delta prediction.

The unit of observation is one RGB image. The intended use is testing compression, reconstruction, and data integrity. The results are not intended for classification or for generalizing compression performance to unrelated datasets.

2.2. Collection and Sampling

The Pistachio Image Dataset was published by Murat Koklu in 2022. It contains 2,148 RGB images of two pistachio varieties:

- Kirmizi: 1,232 images
- Siirt: 916 images

Each image has a resolution of 600 x 600 pixels.

The original images were collected using a computer vision system with a high-resolution camera. According to Singh et al. (2022), pistachio samples were photographed under controlled lighting conditions.

The dataset was originally created for image classification research, distinguishing between the two pistachio varieties.

For P3, we use only the Kirmizi category. The pipeline sorts the filenames and selects the first 10 images.

This is a deterministic sample, not a random sample. It represents only a small part of the full dataset, limiting how broadly our results can be generalized.

While we know the conditions of the samples, we do not know the camera setup, camera used or the sampling rate used to capture the images. This is therefore left as unknown.

2.3. Rights and Privacy

The original dataset is available through Kaggle and Mendeley Data. The Mendeley publication specifies a Creative Commons Attribution 4.0 (CC BY 4.0) licence, permitting reuse and redistribution with appropriate attribution.

The dataset contains photographs of pistachio nuts and does not intentionally contain personal information.

The original dataset is retained separately from the compressed files. No formal retention or deletion schedule has been established for the course project.

2.4. Dataset References

Koklu, M. Pistachio Image Dataset. Kaggle.

Koklu, M. (2022). Pistachio Image Dataset. Mendeley Data, Version 1.

Singh, D. et al. (2022). Classification and Analysis of Pistachio Species with Pre-Trained Deep Learning Models. Electronics, 11(7), 981.

2.5. Labels and Transformations

The selected directory identifies the pistachio category. The original labelling process has not been independently verified.

Images are loaded using Pillow, converted to RGB, and represented as NumPy arrays.

The compression process uses canonical Huffman coding with four possible prediction methods:

- No prediction
- Horizontal delta prediction
- Vertical delta prediction
- Spiral delta prediction

With automatic selection, all four methods are evaluated and the smallest compressed representation is selected.

Delta prediction stores differences between pixel values rather than the original values. Huffman coding then encodes the resulting byte sequence.

The compression is designed to be lossless. The original image arrays can be reconstructed exactly, as tested by comparing the decompressed arrays against the inputs.

Original files are preserved, while compressed versions are stored separately using the `.hcnp` extension.

2.6. Use and Maintenance

The dataset and processing pipeline are maintained by the project group for IDIG4120.

Recommended use: Use with controls.

The dataset is suitable for evaluating the implemented compression pipeline on the selected sample, provided that integrity and reconstruction checks are performed.

3. Schema and Quality Assurance

3.1. Data Schema

The expected format of each loaded image is:

Property	Expected value
Data type	`uint8`
Shape	`(600, 600, 3)`
Colour format	RGB
Pixel range	0–255
Unit of observation	One image

The three dimensions represent image height, width, and colour channels.

The pipeline validates each image against the expected shape, data type, and pixel range.

3.2. Quality Assurance Results

The pipeline checks for missing inputs, invalid image dimensions, unexpected data types, corrupt images, duplicate files, and unusual pixel values.

The following results were obtained from the generated `qa_report.json`:

Measurement	Result
Images processed	10

Manifest entries	10
Corrupt files	0
Failed validation	0
Duplicate originals	0
Observed pixel range	0–255
Lowest image mean	36.37
Highest image mean	47.51

All 10 images passed the implemented validation checks. No corrupt files or duplicate originals were detected in the selected sample.

Pixel values were within the expected range. Nine images had a maximum observed value of 255, while one had a maximum of 249.

The results indicate that the selected images satisfy the defined technical schema. However, these checks do not establish that the original dataset is complete, representative, or correctly labelled.

4. Integrity and Reproducibility

4.1. Manifest Verification

The pipeline generates a manifest at `data/manifest.json`.

For every processed image, the manifest contains:

- Relative paths to original and processed files
- Original and processed file sizes
- SHA-256 hashes for both versions
- Validation results

SHA-256 hashes act as file fingerprints. The verification function recalculates each hash and compares it against the recorded value, allowing missing or modified files to be detected.

The manifest contained 10 entries.

Manifest verification result: PASS.

4.2. Reproducibility

The pipeline uses Python, NumPy, and Pillow. The configuration includes a random seed of 42 and records the versions of the required libraries.

The main pipeline performs the following steps:

1. Load the selected images.
2. Validate image shapes and data types.
3. Compress and save the processed images.
4. Generate the manifest and QA report.
5. Run round-trip tests.
6. Verify manifest integrity.
7. Record execution time and memory usage.

To reproduce the results, another user must obtain the original dataset, preserve the expected project directory structure, install the required dependencies, and execute the notebook in order.

Environment:
- Python: 3.14.8
- Numpy: 2.5.3
- Pillow: 12.3.0

The round-trip tests compare reconstructed arrays with the original input arrays using `np.array_equal()`.

Round-trip verification result: PASS for all tested predictors.

5. Performance and Failure Handling

5.1. Performance

Execution time was measured using Python's `time` module, while peak traced memory allocation was measured with `tracemalloc`.

The recorded results were:

Measurement	Result
Images processed	10
Total execution time	85.93 seconds
Peak traced memory	36.66 MiB
Predictor selection	Automatic
Round-trip tests	PASS
Manifest verification	PASS

The measured execution time includes loading, compression, verification, and reporting.

Automatic prediction selects the smallest compressed representation from the four available methods. This provides a storage-oriented implementation choice at the cost of additional computation.

The figures describe this particular run and should not be interpreted as general performance benchmarks. The traced memory measurement does not necessarily include all memory used by the operating system or native libraries.

P2 also implemented comparisons with PNG and JPEG. PNG was used as a lossless reference, while JPEG was used as a lossy comparison.

5.2. Failure Handling

The pipeline includes handling for several potential failures.

Missing inputs: A missing input directory raises a `FileNotFoundError`.

Digest mismatch: The manifest verification function reports failure if recorded and calculated hashes differ.

Invalid image shape: Images with unexpected dimensions are marked as invalid in the QA results.

Corrupt files: Files that cannot be decoded are recorded and skipped.

Interrupted writes: Output files are written to temporary files before being moved into their final locations using `os.replace()`. This reduces the chance of partially written final files.

These mechanisms are present in the implementation. Their presence does not by itself demonstrate that every failure scenario has been experimentally tested.

6. Limitations and Responsibilities

6.1. Known Limitations

The dataset evaluation uses only 10 images from the Kirmizi category, selected by filename order. This limits the generalizability of the results.

The selected images are also visually similar, as they belong to the same pistachio category and have similar backgrounds, colours, and textures. This may influence compression performance, as repeated patterns and small differences between neighbouring pixels can improve compression efficiency.

Consequently, the measured performance may not be representative of more varied image datasets with different colours, textures, or visual complexity.

Images are converted to RGB before compression. Lossless reconstruction therefore refers to the decoded RGB arrays, not necessarily byte-for-byte reproduction of the original JPEG files.

The QA checks focus on technical consistency and integrity rather than the correctness of labels or collection procedures.

Compression performance and memory usage may vary across hardware, software versions, and image characteristics.

Although the dataset's general collection process and licence are documented, some technical details of the original acquisition setup remain unknown.

6.2. Project Responsibilities

The project was completed jointly by both group members, with responsibilities shared equally throughout the work.

The implementation builds on the codec developed in P2, while P3 focuses on dataset validation, integrity, and reproducibility.

Both members contributed to implementation, testing, and documentation through collaborative development and pair programming.

7. Conclusion

P3 extends the existing compression codec with dataset validation, file integrity verification, and reproducibility documentation.

All 10 selected images passed the implemented quality checks. Manifest verification and round-trip reconstruction tests also passed.

The main limitations are the small sample size, the visual similarity between the selected images, and the limited information available about the original image acquisition setup.

The accompanying notebook, manifest, and QA report provide the technical evidence needed to inspect and reproduce the processing pipeline.

8. AI Declaration

NTNU

Declaration of AI aids and tools

Have any AI-based aids or tools been used in the creation of this report?

✓ No Yes

If yes: please specify the aid/tool and area of use below.

Text

Spell checking. Are parts of the text checked by: Grammarly, Ginger, Grammarbot, LanguageTool, ProWritingAid, Sapling, Trinka AI or similar tools?

✓ **Text-generation.** Are parts of the text generated by: ChatGPT, GrammarlyGO, Copy.AI, WordAi, WriteSonic, Jasper, Simplified, Rytr or similar tools?

Writing assistance. Are one or more of the report's ideas or approach suggested by: ChatGPT, Google Bard, Bing chat, YouChat or similar tools?

If no, sign document. If yes, use of text aids/tools apply to this report -please specify usage here:

The document has only been language washed to help formulate things easier

---

Codes and algorithms

✓ **Programming assistance.** Are parts of the codes/algorithms that i) appear directly in the report or ii)have been used to produce results such as figures, tables or numerical values been generated by: GitHub Copilot, CodeGPT, Google Codey/Studio Bot, Replit Ghostwriter, Amazon CodeWhisperer, GPTEngineer, ChatGPT, Google Bard or other tools.

If yes, use of programming assistance aid/tools apply to this report - please specify usage here:

AI has been used as a guider rather than a generator, helping us as we write the code

---

Images and figures

Image generation. Are one or more of the reports images/figures generated by: Midjourney, Jasper, WriteSonic, Stability AI, Dall-E or similar tools?

If yes, use of image generator aids/tools apply to this report – please specify usage here:

Other AI aids or tools. Have you used other types of AI aids or tools in the creation of this report?
If yes, please specify usage here:

✓ I am familiar with NTNU's regulations on artificial intelligence. I declare that any use of AI aids or tools are explicitly stated i) directly in the report or ii) in this declaration form.

09-11-2026/NTNU In Gjøvik

Signature/Date/Place

Ole Marius Glomsrud
Vinjar Aurstad Berland